

Conceptos fundamentales en aprendizaje de máquina

En esta guía se van a explicar los conceptos fundamentales en aprendizaje de máquina, conceptos que son esenciales que cualquier ingeniero de *machine learning* o científico de datos domine, para que pueda construir modelos predictivos que sean realmente funcionales.

Aunque la guía se va a referir a modelos de regresión lineal, los conceptos que se van a explicar son universales, es decir que se aplican igual a cualquier modelo de aprendizaje de máquina, bien sea de regresión o de clasificación.

¿Cómo aprende un modelo de aprendizaje de máquina?

La forma en que **aprende** cualquier modelo de aprendizaje de máquina supervisado, cómo lo son los modelos de regresión y clasificación, se muestra en la Figura 1.

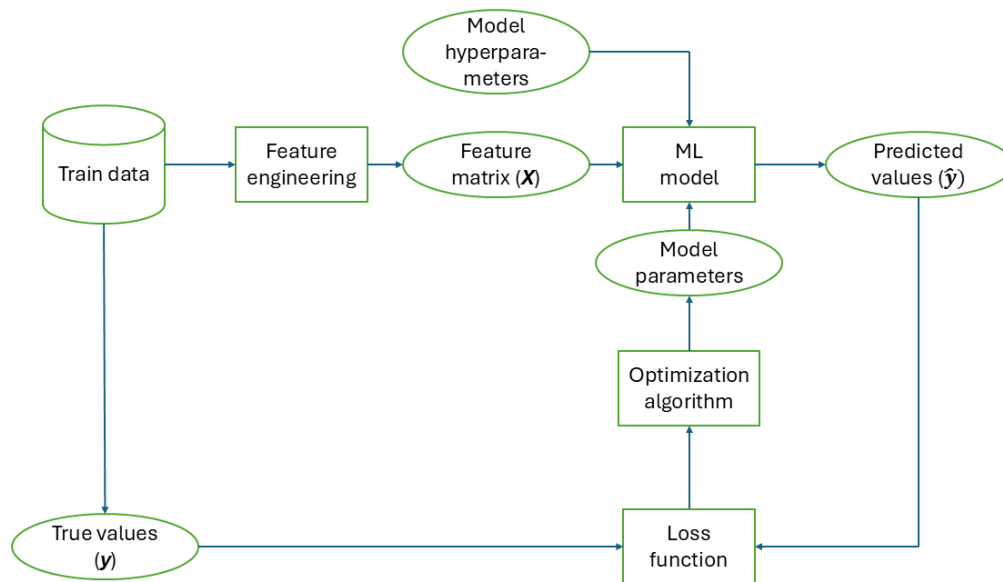


Figura 1. Proceso de aprendizaje de un modelo de aprendizaje de máquina

En términos simples, un modelo de aprendizaje de máquina toma un conjunto de características, conocidas como **matriz de características (X)**, que han sido obtenidas luego de procesar un conjunto de **datos de entrenamiento**, y con esta matriz de características hace predicciones sobre otra variable, conocida como **variable objetivo**. El modelo intentará que las **predicciones ($\hat{y}$)** que hace sean lo más parecidas posible a los **valores reales (y)** de la variable que se quiere predecir. Para esto, deberá existir alguna función matemática, llamada **función de costo o de**

pérdida, que mida la diferencia entre lo que el modelo predice y los valores reales; esta diferencia se llama de manera general **error**. El modelo tendrá implementado algún método de **optimización** que le permita minimizar esta función de pérdida, y por ende el error del modelo, y como resultado de este proceso de optimización algunos **parámetros** del modelo se modificarán o ajustarán. En este punto, se dice que el modelo ya está **entrenado o ajustado**, y se puede usar para hacer predicciones sobre **datos nuevos** con él.

Los modelos de aprendizaje de máquina suelen tener otros parámetros que no son optimizados en el proceso de entrenamiento descrito anteriormente; para diferenciar estos parámetros de los que sí son optimizados, a los que no son optimizados se les llama **hiperparámetros**.

Modelos de regresión lineal

Un modelo de regresión lineal simple busca predecir un valor de salida continuo, a partir de una o más características de entrada (usualmente llamadas variables predictoras), asumiendo una relación **lineal**. La ecuación básica de un modelo de regresión lineal simple es:

$$y = w_0 + w_1 * x_1 + w_2 * x_2 + \dots + w_p * x_p$$

donde w_0 es el intercepto o término independiente, y w_p es la pendiente, coeficiente o peso asociado a la característica x_p . Así, en este modelo, cada variable tendrá un coeficiente asociado.

En el caso de estos modelos, los parámetros que se optimizan son los pesos o coeficientes asociados a las características del modelo.

Función de costo (o pérdida)

La función de costo (también conocida como función de pérdida o función de error) es una medida matemática que cuantifica la discrepancia entre los valores predichos por el modelo y los valores reales.

Su objetivo principal es evaluar el rendimiento del modelo y guiar el ajuste de sus parámetros para minimizar el error **durante el entrenamiento**.

Para los modelos de regresión lineal, las funciones de costo más comunes son el Error Cuadrático Medio (MSE) y la Suma de Cuadrados de los Residuos (RSS).

- **RSS** es la suma de los cuadrados de las diferencias entre los valores reales y los valores que predice el modelo.

- **MSE** es el **promedio** de las diferencias al cuadrado entre los valores predichos y los valores reales.

La meta del entrenamiento es que el valor de la función de costo sea lo más cercano a cero posible.

Optimización de la función de costo

Optimizar (minimizar en el caso de los modelos de regresión) la función de costo tiene como objetivo principal encontrar los parámetros (pesos o coeficientes) del modelo que resulten en el menor error posible entre las predicciones del modelo y los valores reales.

Existen dos enfoques principales para solucionar este problema de optimización en regresión lineal:

Minimización por mínimos cuadrados:

Este método busca directamente los valores de los parámetros que minimizan la función de costo (RSS o MSE). Es una solución analítica que **no requiere iteraciones**.

Se logra derivando la función de costo con respecto a cada uno de los parámetros del modelo e igualando esas derivadas a cero. Esto es posible porque las funciones de costo como el MSE y el RSS son **convexas y diferenciables**, lo que garantiza que existe un único mínimo global.

Para la regresión lineal simple, las fórmulas explícitas para los pesos se obtienen al igualar las derivadas parciales a cero. Para la regresión múltiple, el modelo se puede expresar en notación matricial y la solución de forma cerrada es:

$$\widehat{W} = (X^T X)^{-1} X^T y$$

Aunque es directo, calcular la inversa de una matriz en la solución de forma cerrada puede ser **computacionalmente muy demandante** y depende de que el número de observaciones sea mayor que el número de características, por lo que a menudo se prefiere el método de descenso de gradiente para conjuntos de datos grandes.

Descenso de gradiente:

Es un algoritmo de optimización iterativo de primer orden ampliamente utilizado para entrenar modelos de machine learning y redes neuronales, incluyendo, pero no limitándose, a la regresión lineal.

La idea es encontrar el mínimo local (o global en el caso de funciones convexas) de una función **diferenciable**. Para ello, el algoritmo toma **pasos repetidos en la dirección**

opuesta al gradiente (la dirección del descenso más pronunciado) de la función de costo (ver Figura 2).

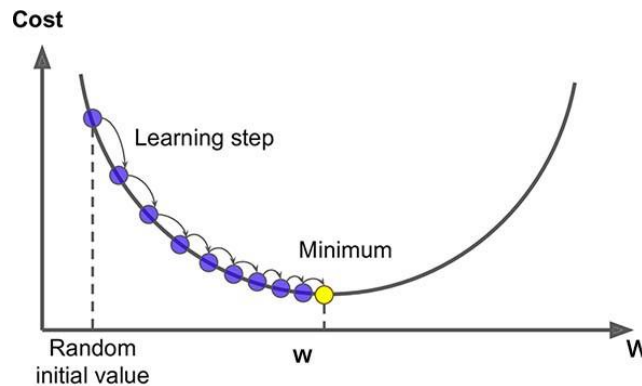


Figura 2. Descenso de gradiente

El algoritmo para una sola variable funciona de la siguiente manera:

1. Se elige un punto de partida inicial (valores para los parámetros w_0 , w_1 , etc.) de manera aleatoria.
2. En cada iteración, se calcula el gradiente (derivada) de la función de costo en el punto actual. Este gradiente indica tanto la dirección como la magnitud del cambio que debe realizarse.
3. Los parámetros se actualizan utilizando la fórmula:

$$w^{(t-1)} \leftarrow w^{(t)} - \eta \left. \frac{dg}{dw} \right|_{w^{(t)}}$$

Donde η la **tasa de aprendizaje**, que es un **hiperparámetro** que determina el tamaño de los pasos. Una tasa pequeña puede hacer que la convergencia sea lenta, mientras que una tasa muy grande puede hacer que el algoritmo "salte" el mínimo y no converja.

4. El proceso se detiene cuando se alcanza un número máximo de iteraciones predefinido, o cuando el tamaño del paso (o la magnitud del gradiente) es menor que un valor de tolerancia establecido, indicando que se ha llegado al mínimo o se está muy cerca de él.

En un modelo con más de una característica, los pesos asociados a estas se actualizan de manera simultánea, de acuerdo con la siguiente ecuación matricial:

$$\mathbf{W}^{(t+1)} = \mathbf{W}^{(t)} + 2\eta \mathbf{X}^T (\mathbf{y} - \mathbf{XW}^{(t)})$$

Evaluación de modelos de aprendizaje de máquina

La evaluación de modelos es el proceso de cuantificar el rendimiento de un modelo de aprendizaje de máquina. El objetivo principal es determinar si el modelo es lo suficientemente bueno para el propósito que fue creado y si puede **generalizar** a nuevos datos.

Generalizar en el contexto del aprendizaje automático significa que un modelo tiene la capacidad de hacer predicciones precisas en datos **que no ha visto antes**. Para entender este concepto piense en un estudiante que está preparando un examen. Si el estudiante solo memoriza las respuestas a las preguntas del libro de texto (los datos de entrenamiento), podría obtener una puntuación perfecta en una prueba con preguntas idénticas. Sin embargo, si el examen tiene preguntas nuevas (datos de prueba) basadas en los mismos conceptos, el estudiante fracasará si no ha aprendido a generalizar. En el aprendizaje automático, el objetivo no es que el modelo memorice los datos de entrenamiento, sino que aprenda los patrones y las relaciones subyacentes.

Métrica de desempeño o evaluación

Una métrica de desempeño (o métrica de evaluación) es una medida que se utiliza para evaluar qué tan bien se desempeña un modelo predictivo. Su propósito final es comprender cómo funcionará un modelo con datos que nunca ha visto, es decir, su **capacidad de generalización**.

Estas métricas se utilizan **después del entrenamiento** del modelo para ofrecer una medida más intuitiva y comprensible de su rendimiento. La elección de la métrica adecuada depende del tipo de problema que se esté abordando (clasificación, regresión o agrupación) y las características específicas del conjunto de datos. Para problemas de regresión (salidas continuas) se utilizan como métricas

- **Error Cuadrático Medio (MSE):** Calcula el promedio de las diferencias al cuadrado entre los valores predichos y reales.
- **Raíz del Error Cuadrático Medio (RMSE):** Es la raíz cuadrada del MSE, expresando el error en las mismas unidades que la salida.
- **Error Absoluto Medio (MAE):** Media de las diferencias absolutas. Es más resistente a valores atípicos que el MSE.
- **Pérdida de Huber (Huber Loss):** Combina MSE y MAE para ser robusta a valores atípicos y, a la vez, diferenciable.

- **Coefficiente de Determinación (R^2):** Indica la proporción de la varianza en la variable dependiente que es predecible a partir de las independientes. **Nota:** No se recomienda utilizar esta métrica.

Diferencias clave entre funciones de costo y métricas de evaluación

La distinción fundamental entre una función de costo y una métrica de desempeño radica en su **propósito y momento de uso**:

1. **Propósito principal:** La función de costo guía la optimización y el aprendizaje del modelo durante el entrenamiento, indicándole cómo ajustar sus parámetros para minimizar el error. La métrica de desempeño evalúa el rendimiento final del modelo en datos que no ha visto, proporcionando una comprensión de su capacidad para generalizar a situaciones del mundo real.
2. **Momento de uso:** La función de costo se utiliza durante el proceso de entrenamiento para ajustar los parámetros del modelo. Las métricas de desempeño se utilizan después del entrenamiento para evaluar el modelo ya ajustado.
3. **Requisito de diferenciabilidad:** La función de costo debe ser diferenciable para que los algoritmos de optimización basados en gradientes puedan operar. Las métricas de desempeño, en cambio, no siempre necesitan ser diferenciables y se enfocan más en la interpretación y comprensión humana del rendimiento.

Errores de entrenamiento, prueba y generalización

Los errores de entrenamiento, prueba y generalización son conceptos fundamentales en el aprendizaje automático que permiten evaluar la capacidad de un modelo para realizar predicciones precisas y su utilidad en escenarios del mundo real.

A continuación, se detalla cada uno:

Error de entrenamiento

Es el error que el modelo comete en los datos con los que fue **entrenado**. En otras palabras, mide qué tan bien el modelo predice o clasifica en el conjunto de datos que ya ha visto. Este error se calcula como la pérdida promedio sobre los datos de entrenamiento.

El error de entrenamiento por sí solo **no es un buen indicador** de la capacidad del modelo para hacer buenas predicciones en datos nuevos, ya que los parámetros del

modelo se ajustan específicamente a estos datos, lo que puede dar una medida optimista de su desempeño.

Error de prueba

Es el error que el modelo comete al predecir sobre datos que no ha visto antes y que no fueron incluidos en el conjunto de entrenamiento.

El error de prueba permite aproximar el error de generalización. Se utiliza para estimar la precisión con la que el modelo se desempeñará en entradas previamente no observadas.

En un escenario ideal, el conjunto de prueba y el conjunto de entrenamiento deben estar completamente separados, para evitar que el modelo “aprenda” de los datos de prueba, lo que se conoce como **filtración de datos**.

Error de generalización

Es una medida de la precisión con la que un algoritmo es capaz de predecir resultados para datos nunca vistos. Un modelo con buen desempeño debería tener un error de generalización bajo, que indica que el modelo ha aprendido patrones generales y los aplica correctamente a datos no vistos, lo que lo convierte en una herramienta útil y robusta.

Formalmente, el error de generalización se define como el valor esperado del error de prueba en todas las observaciones posibles. Depende de la distribución de probabilidad conjunta desconocida de las entradas y salidas.

El error de generalización no se puede calcular directamente debido a que la distribución de probabilidad conjunta es desconocida. Sin embargo, se puede aproximar evaluando el modelo con varios conjuntos de prueba, a menudo utilizando técnicas como la **validación cruzada**.

En esencia, la meta en el aprendizaje automático es desarrollar modelos con un bajo error de generalización, es decir, modelos que aprendan patrones que son verdaderamente representativos de los datos subyacentes y no solo las particularidades del conjunto de entrenamiento.

Validación cruzada

La validación cruzada (K-fold cross-validation) es un método para estimar el rendimiento de un modelo de manera más fiable que con una simple división de datos de entrenamiento y prueba. Es especialmente útil cuando se tiene un conjunto de datos limitado. Funciona de la siguiente manera:

1. El conjunto de datos de entrenamiento se divide en **K subconjuntos** (o *folds*) de igual tamaño.
2. El modelo se entrena **K veces**. En cada iteración:
 - Uno de los folds se usa como conjunto de validación.
 - Los K-1 folds restantes se usan para entrenar el modelo.
 - Se calcula el error del modelo en el fold de validación.
3. Al final de las K iteraciones, se promedia el error de validación de cada fold. Este promedio es la métrica de rendimiento final del modelo.

La validación cruzada permite hacer una evaluación más precisa de los modelos, ya que, al usar diferentes subconjuntos de datos para la evaluación, nos da una idea más precisa de cómo se comportará el modelo con datos no vistos. Además, esta técnica hace un uso eficiente de los datos, ya que cada observación del conjunto de datos se utiliza tanto para el entrenamiento como para la evaluación. Esto es ideal cuando se tienen pocos datos.

Sesgo y varianza, subajuste y sobreajuste

En el contexto del aprendizaje de máquina, el sesgo (bias) y la varianza (variance) son dos fuentes de error fundamentales que afectan la capacidad de un modelo para generalizar a datos nuevos. Se dará una breve explicación de que son, y cómo afectan el desempeño de los modelos.

Sesgo

El sesgo se refiere a los supuestos que hace un modelo para aprender la función objetivo. Un modelo con alto sesgo tiende a simplificar demasiado el problema, lo que puede llevar a errores sistemáticos. Por ejemplo, usar una regresión lineal para datos que claramente siguen una curva no lineal. De esta manera, un **sesgo alto** significa que el modelo es **demasiado simple** y no es capaz de capturar las relaciones y patrones complejos en los datos de manera adecuada. Esto lleva a que el modelo no se ajuste bien ni siquiera a los datos de entrenamiento, resultando en un **subajuste (underfitting)**. Los modelos complejos, por otro lado, tienden a tener un menor sesgo. Un sesgo alto se traduce en un **error de entrenamiento elevado**.

Varianza

La varianza mide cuánto cambia el modelo cuando se entrena con diferentes subconjuntos de datos. Un modelo con alta varianza se ajusta demasiado a los datos de entrenamiento y puede fallar al generalizar. Por ejemplo, un árbol de decisión muy profundo que memoriza los datos de entrenamiento. Una **varianza alta** indica que el modelo es **demasiado sensible** a las particularidades de los datos de entrenamiento, incluyendo el ruido o las peculiaridades que no son generalizables. Esto lleva a un excelente desempeño en los datos de entrenamiento, pero un rendimiento pobre en datos de prueba, lo que se conoce como **sobreajuste (overfitting)**. Los modelos complejos tienden a tener una mayor varianza. Una varianza elevada implica un **alto nivel de error durante las pruebas y la validación, pero bajo durante el entrenamiento**.

Relación entre sesgo, varianza y el error de generalización

El sesgo, la varianza y el error de generalización están íntimamente conectados a través de la **compensación sesgo-varianza (*bias-variance trade-off*)**. El error total de un modelo, a menudo aproximado por el Error Cuadrático Medio (MSE), puede descomponerse en la suma de la varianza, el sesgo al cuadrado y un término de ruido irreducible:

$$\text{MSE} = \text{Varianza} + \text{Sesgo}^2 + \text{Ruido}$$

Esta relación ilustra cómo el sesgo y la varianza contribuyen al error de predicción del modelo.

- **Subajuste (alto sesgo, baja varianza):** Un modelo muy simple tendrá un sesgo alto, ya que no puede representar adecuadamente la complejidad de los datos, lo que resulta en errores altos tanto en el entrenamiento como en los datos de prueba. Al ser simple, su varianza es baja, es decir, el modelo no cambia mucho si se entrena con diferentes subconjuntos de datos. El subajuste lleva a un **error de generalización alto**.
- **Sobreajuste (bajo sesgo, alta varianza):** Un modelo excesivamente complejo, por otro lado, puede ajustarse demasiado bien a los datos de entrenamiento, capturando el ruido y las peculiaridades específicas de ese conjunto. Esto reduce el sesgo en el conjunto de entrenamiento, pero aumenta la varianza, haciendo que el modelo sea muy sensible a los datos de entrenamiento y rinda mal en datos no vistos. El sobreajuste es una de las principales causas de un **error de generalización alto**.

El desafío es que **es imposible minimizar ambos (sesgo y varianza) simultáneamente**. Reducir el sesgo (haciendo el modelo más complejo) tiende a

aumentar la varianza, y viceversa. El objetivo es encontrar un **equilibrio óptimo** entre sesgo y varianza para minimizar el error de generalización. Técnicas como la regularización (por ejemplo, L1 y L2), el aumento del tamaño del conjunto de datos y la validación cruzada ayudan a gestionar esta compensación y a reducir el error de generalización.

Técnicas para reducir el sobreajuste

El **sobreajuste** ocurre cuando un modelo aprende demasiado bien los detalles y el ruido de los datos de entrenamiento, hasta el punto de que pierde su capacidad para **generalizar** a datos nuevos e inéditos. Esto se manifiesta en un excelente desempeño con los datos de entrenamiento, pero una caída drástica en la precisión cuando se enfrenta a datos no vistos. La brecha entre el error de entrenamiento y el error de prueba se vuelve demasiado grande.

Para evitar el sobreajuste, se pueden aplicar diversas estrategias y técnicas:

1. **Regularización:** La regularización introduce penalizaciones en los coeficientes del modelo para evitar que crezcan excesivamente. Esto compensa una disminución marginal en la precisión del entrenamiento con un aumento en la generalización, disminuyendo la varianza del modelo a costa de un mayor sesgo.
 - **Regularización L1 (Lasso):** Reduce ciertos parámetros a cero, promoviendo la selección de variables relevantes y generando soluciones más dispersas.
 - **Regularización L2 (Ridge o Tikhonov):** Reduce el impacto de parámetros extremos sin eliminarlos por completo, acercando sus magnitudes a cero, pero sin llegar a ellas.
2. **Aumentar el tamaño del conjunto de datos:** Cuantos más datos se tengan para entrenar, más probable será que el modelo aprenda patrones generales en lugar de detalles específicos del conjunto de entrenamiento. Esto reduce el riesgo de sobreajuste y mejora la capacidad de generalización.
3. **Aumento de datos (Data Augmentation):** Cuando no es posible obtener más datos reales, el aumento de datos genera nuevas muestras de entrenamiento a partir de las existentes (por ejemplo, rotaciones de imágenes) para aumentar la diversidad del conjunto de entrenamiento.
4. **Uso de validación cruzada:** Es un método de remuestreo que permite evaluar un modelo de manera más robusta, incluso con datos limitados. Divide los

datos en múltiples subconjuntos para entrenar y probar el modelo varias veces, lo que proporciona una estimación más precisa del rendimiento general y reduce el sobreajuste.

5. **Detención temprana (Early Stopping):** Consiste en detener el entrenamiento del modelo cuando su rendimiento en el conjunto de validación comienza a empeorar, incluso si el error de entrenamiento sigue disminuyendo. Esto evita que el modelo se sobreajuste al detener el proceso en el punto de mejor generalización. En redes neuronales, puede interpretarse como una forma de regularización L2.
6. **Reducción de la complejidad del modelo:** Un modelo excesivamente complejo puede capturar patrones irrelevantes o ruido. Simplificar el modelo, reduciendo el número de parámetros o capas, puede mejorar su capacidad de generalización. Una estrategia relacionada es la **selección de características**, que implica eliminar variables irrelevantes o redundantes para que el modelo se concentre en la información más importante.
7. **Inyección de ruido en los datos (Noise Injection):** Agregar una pequeña cantidad de ruido a los datos de entrada o a los pesos del modelo durante el entrenamiento puede ayudarlo a generalizar mejor, evitando que se ajuste demasiado a patrones específicos. En algunos casos, esto puede interpretarse como una regularización L2 en las segundas derivadas de las salidas de la red.

Modelos de regresión lineal regularizados: Ridge y Lasso

En los modelos de regresión lineal, a medida que la complejidad del modelo aumenta, la varianza tiende a crecer mientras que el sesgo disminuye. El objetivo es encontrar el punto óptimo donde el error de generalización sea mínimo.

Para reducir la complejidad del modelo, una técnica común es penalizar el tamaño de los coeficientes. La función de costo de un modelo regularizado se define como la suma de una medida del error y una medida de la magnitud de los coeficientes.

Regresión Ridge

La regresión Ridge utiliza la norma L2 para penalizar los coeficientes. La función de costo para la regresión Ridge se expresa de la siguiente manera:

$$L(y, f(\hat{w})) = MSE(\hat{w}) + \lambda \|w\|_2^2$$

Donde:

- $MSE(\hat{w})$ es la medida del error.

- λ es el hiperparámetro de regularización, con un valor que puede variar de 0 a ∞ .
- $\|w\|^2$ es la medida del tamaño de los parámetros, que en este caso se calcula usando la norma L2 de los coeficientes, que es la suma de los cuadrados de estos.

En forma matricial, la función de costo es:

$$L(y, f(\hat{W})) = (y - XW)^T (y - XW) + \lambda W^T W$$

La función de costo de Ridge es derivable, lo que permite encontrar una solución de forma cerrada para los coeficientes:

$$\hat{W}^{ridge} = (X^T + \lambda I)^{-1} X^T y$$

Ridge también se puede optimizar mediante el algoritmo de descenso de gradiente. La penalización de la norma L2 reduce proporcionalmente la magnitud de los coeficientes hacia cero, pero no los fuerza a ser exactamente cero.

Regularización Lasso

La regresión Lasso (*Least Absolute Shrinkage and Selection Operator*) utiliza la norma L1 para penalizar los coeficientes, lo que a menudo resulta en coeficientes que son exactamente cero. La función de costo de Lasso es:

$$L(y, f(\hat{w})) = MSE(\hat{w}) + \lambda \|w\|_1$$

Donde:

- $MSE(\hat{w})$ es la medida del error.
- $\|w\|_1$ es la norma L1 de los coeficientes, que es la suma de los valores absolutos de los mismos.
- λ es el hiperparámetro de regularización, con un valor que puede variar de 0 a ∞ .

A diferencia de la regresión Ridge, la función de costo de Lasso no es derivable en todos los puntos. Esto significa que no tiene una solución de forma cerrada y que tampoco se puede usar el algoritmo de descenso de gradiente convencional. En su lugar, se utiliza un algoritmo de optimización llamado **descenso de coordenadas**, que minimiza la función de costo una coordenada (característica) a la vez.

Comparación entre Ridge y Lasso

- **Penalización:** Ridge usa la norma L2, mientras que Lasso usa la norma L1.
- **Selección de características:** Ridge reduce la magnitud de los coeficientes, pero rara vez los lleva a cero. Lasso, debido a la penalización L1, puede forzar a algunos coeficientes a ser exactamente cero, lo que lo convierte en un método de selección de características intrínseco.
- **Optimización:** La función de costo de Ridge se puede optimizar de forma cerrada o por descenso de gradiente, mientras que Lasso utiliza el algoritmo de descenso de coordenadas.

Sintonización de hiperparámetros con validación cruzada

El valor del hiperparámetro λ es crucial para el rendimiento del modelo regularizado y debe ser seleccionado cuidadosamente. Cuando se tienen muchos datos, se puede dividir el conjunto de datos en un conjunto de entrenamiento, un conjunto de validación, y un conjunto de prueba. El conjunto de validación se utiliza para seleccionar el valor óptimo de λ que minimiza el error, y luego el conjunto de prueba se usa para evaluar el error de generalización final del modelo seleccionado.

Si se tienen menos datos, la validación cruzada es una técnica más robusta. El procedimiento de validación cruzada (K-fold cross-validation) es el siguiente:

1. El conjunto de datos de entrenamiento se divide en K particiones (folds) de tamaño N/k , donde N es el número total de datos.
2. Para cada partición $k = 1, \dots, K$:
 - Se utiliza una partición como conjunto de validación y las K-1 restantes como conjunto de entrenamiento.
 - Se entrena el modelo con un λ dado, usando las particiones de entrenamiento, y se estiman los parámetros del modelo $\hat{W}_\lambda^{(k)}$.
 - Se calcula el error en el bloque de validación: $error_k(\lambda)$.
3. Se calcula el error promedio de todas las particiones para un valor específico de λ . Esto se conoce como el error de validación cruzada:

$$CV(\lambda) = \frac{1}{K} \sum_{k=1}^K error_k(\lambda)$$

4. Este procedimiento se repite para diferentes valores de λ .

5. Finalmente, se elige el valor de λ que minimiza el error de validación cruzada, $CV(\lambda)$.

Este proceso permite encontrar el valor óptimo del hiperparámetro sin necesidad de un conjunto de validación separado, aprovechando al máximo los datos disponibles para el entrenamiento y la sintonización del modelo.